МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»»

Институт компьютерных наук
Кафедра инженерной кибернетики

ОТЧЁТ
по лабораторной работе №2
по дисциплине «Современные интеллектуальные сетевые сервисы»

Развитие AI-сервиса: сквозная идентификация запросов, асинхронная обработка и интеграция LLM

Выполнил:
студент группы МПИ-25-1-1
Сульповар Михаил Леонидович

Проверил:
М. Е. Корнет

Москва, 2026

1. Цель работы

Расширить сервис агроскоринга из лабораторной работы №1: добавить сквозную идентификацию запросов, постоянное хранение прогнозов, асинхронную пакетную обработку и внешний LLM-компонент. Проверить контракт API, обработку отказов, ограничение очереди и таймауты.

2. Архитектура и исходный код

Проект продолжает `https://github.com/sulpovar-m/agro-scoring-api`. Сохранены входные поля, справочник регионов и правила модели версии 1.1, включая поправку за экстремальную температуру. В соответствии с новым ТЗ успешный `/predict` возвращает 200 после записи в SQLite.

Адрес репозитория для размещения работы: `https://github.com/sulpovar-m/agro-scoring-async-api`.

Компонент	Ответственность
`app/main.py`	Lifespan, middleware, обработчики ошибок.
`app/api/routes.py`	Маршруты и параметры HTTP.
`app/schemas/`	Pydantic-контракты запросов и ответов.
`app/models/risk.py`	Учебная модель на правилах.
`app/services/scoring.py`	Бизнес-валидация, вызов модели, сохранение.
`app/services/batching.py`	Очередь, worker, индивидуальные Future.
`app/services/llm.py`	Клиент OpenAI-совместимого провайдера.
`app/repositories/`	Асинхронные SQL-операции.

Путь обычного запроса: middleware, маршрут, сервис скоринга, модель, SQLite, HTTP-ответ. Для queued-запроса перед моделью добавляется очередь и worker. LLM вызывается отдельно после получения числового прогноза. Соединения и фоновая задача создаются и закрываются в lifespan.

Среда проверки: Python 3.12, FastAPI 0.162.0, Uvicorn 0.54.0, aiosqlite 0.22.1, httpx 0.28.1. Запуск:

```
python –m pip install –r requirements.txt
python –m uvicorn app.main:app ––reload
```

SQLite создаётся автоматически. Дополнительная настройка БД и API-ключ не требуются. Используется один worker: динамическая очередь находится в памяти процесса.

3. Swagger UI

Интерфейс доступен по `http://127.0.0.1:8000/docs`, описание OpenAPI — по `/openapi.json`. Маршруты сгруппированы по назначению: состояние системы, скоринг, история и LLM. Заголовок `x-request-id` доступен для ввода в Swagger.

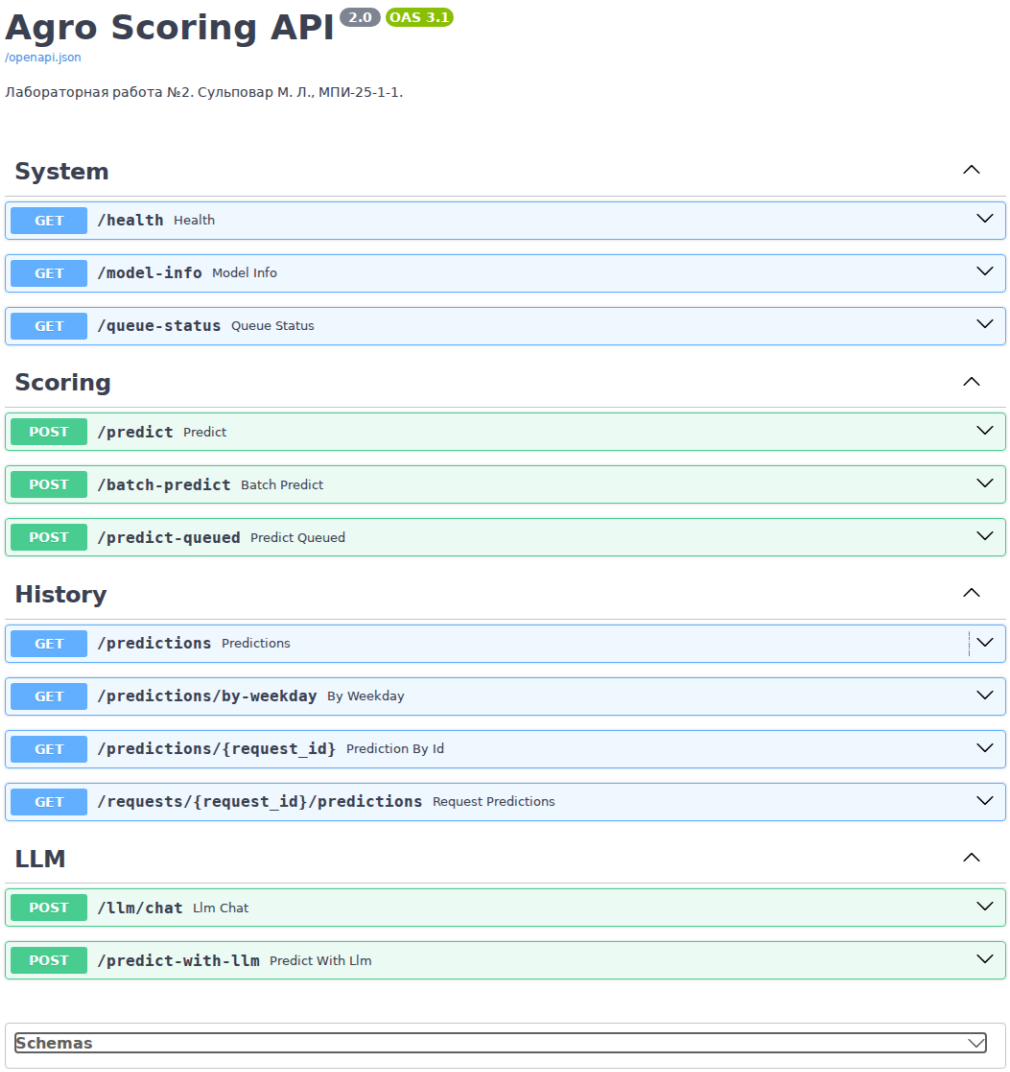


Рис. 1: Swagger UI работающего приложения

4. Сквозной request_id и ошибки (задания 1–3)

Middleware использует переданный `X-Request-ID` или создаёт UUID. Идентификатор записывается в `request.state.request_id`, проходит в результат и SQLite, появляется в логах и заголовке ответа. `X-Process-Time` измеряется через `time.perf_counter()` и выражается в секундах.

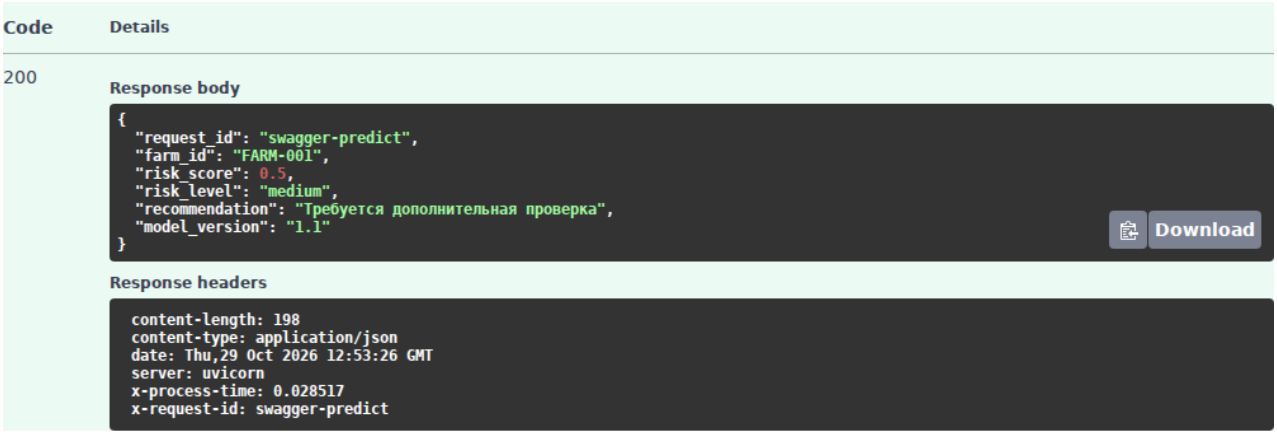


Рис. 2: POST /predict: сохранённый результат, идентификатор и заголовки

Для вымышленного хозяйства из `examples/farm.json` результат равен $0.1 + 0.3 + 0.1 = 0.5$: просрочка больше 30 дней и осадки меньше 100 мм. Категория — medium. Обработчик возвращает 200 только после успешного commit.

Код	Причина
400	Регион или день недели отсутствует в бизнес-справочнике.
404	Прогнозы по идентификатору не найдены.
409	По идентификатору найдено несколько прогнозов.
422	Нарушены ограничения Pydantic или query-параметров.
429	Очередь ожидающих запросов заполнена.
500	Непредвиденная внутренняя ошибка.
502	HTTP-ошибка провайдера или некорректный формат ответа LLM.
503	Недоступны модель, БД, worker или внешнее соединение; HTTP 503 провайдера.
504	Превышен таймаут модели или LLM.

Во всех этих ответах возвращаются `detail` и `request_id`; заголовки добавляются также к ошибкам. При 500 подробности остаются в журнале, клиент получает общее сообщение.

5. SQLite и чтение результатов (задания 4–6)

В файле `agro_scoring.db` создаётся таблица `predictions`. Поля: `id`, `request_id`, `farm_id`, `risk_score`, `risk_level`, `recommendation`, `model_version`, `source`, `created_at`. Идентификатор записи — целочисленный первичный ключ. По request_id создаётся неуникальный индекс: один batch содержит несколько записей с общим идентификатором.

Дата формируется SQLite в UTC. Поле source различает обычный, явный пакетный, очередной прогноз и прогноз с объяснением: single, batch, queued, llm. SQL-параметры передаются отдельно от текста запросов.

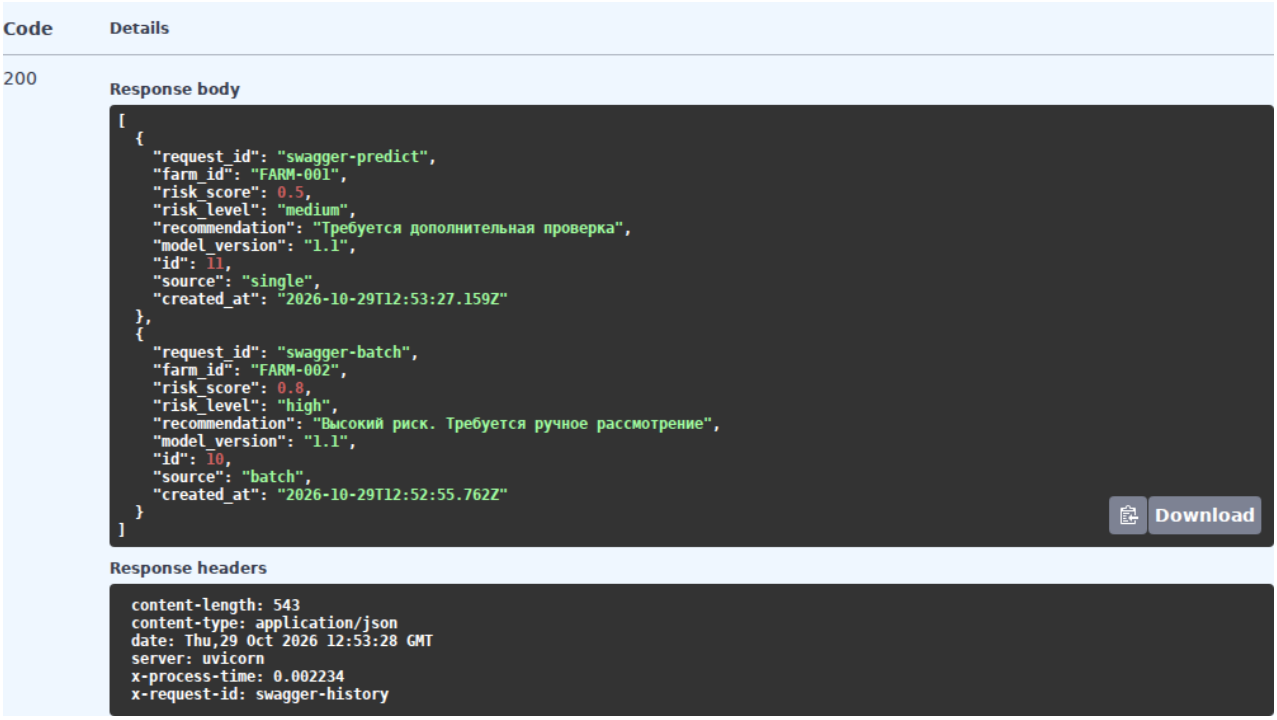


Рис. 3: GET /predictions?limit=2: история прочитана из SQLite

`GET/predictions` поддерживает ограничение количества записей и фильтр категории. `GET/predictions/{request_id}` возвращает одну запись, 404 при отсутствии и 409 при нескольких. `GET/requests/{request_id}/predictions` возвращает весь пакет. Новые записи идут первыми по id; внутри одного запроса записи читаются по возрастанию id.

Почему не sqlite3.execute() внутри async endpoint? Синхронная операция блокирует поток event loop на время дискового ввода-вывода или ожидания блокировки. aiosqlite выполняет работу SQLite в отдельном потоке и позволяет ожидать результат через await. Проверка перезапуска подтверждает, что история не зависит от словаря Python и сохраняется на диске.

6. Пагинация по дню недели (задание 7)

`GET/predictions/by-weekday` принимает weekday, page и per_page. Дни недели сопоставляются значениям `strftime('%w', created_at)`. Сначала выполняется COUNT(*), затем SELECT с LIMIT и OFFSET. Оба чтения выполняются в одной транзакции. Смещение равно $(page - 1) \cdot per_page$.

Ответ содержит data и meta: weekday, current_page, per_page, total_pages, total_items, next_page, prev_page. При неизвестном дне возвращается 400, при отрицательной странице — 422. Пустая страница корректно возвращает 200 и пустой data.

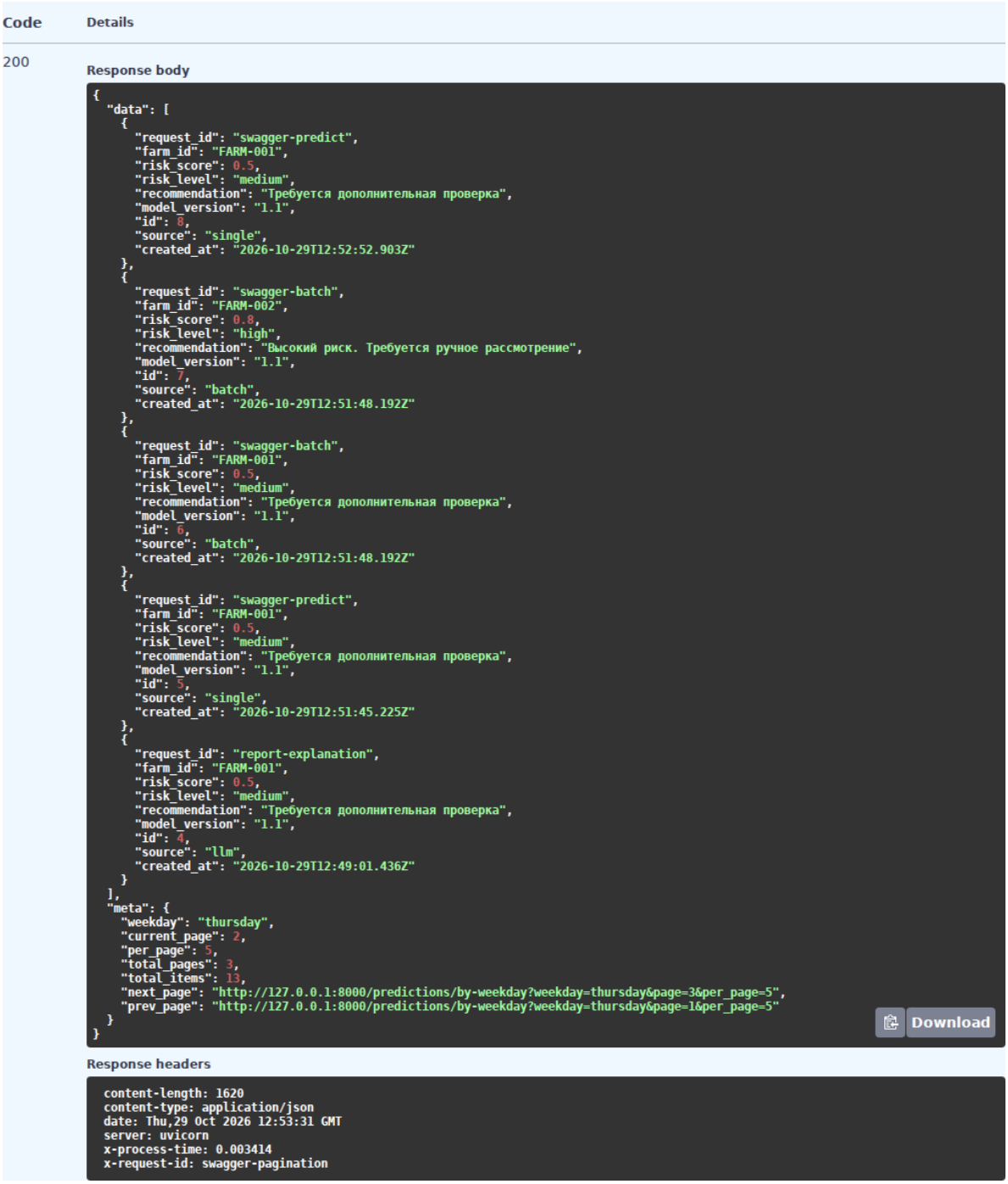


Рис. 4: Пагинация по четвергу: page=2, per_page=5; видны записи, счётчики и ссылки

7. Явный пакет и состояние очереди (задания 8, 10)

`POST/batch-predict` принимает список items длиной от 1 до 8. Весь пакет проходит бизнес-валидацию до вызова модели. Выполняется один `model_predict_batch()`, затем один `executemany()` и один commit. Все записи имеют общий request_id, а каждый элемент содержит собственный farm_id.

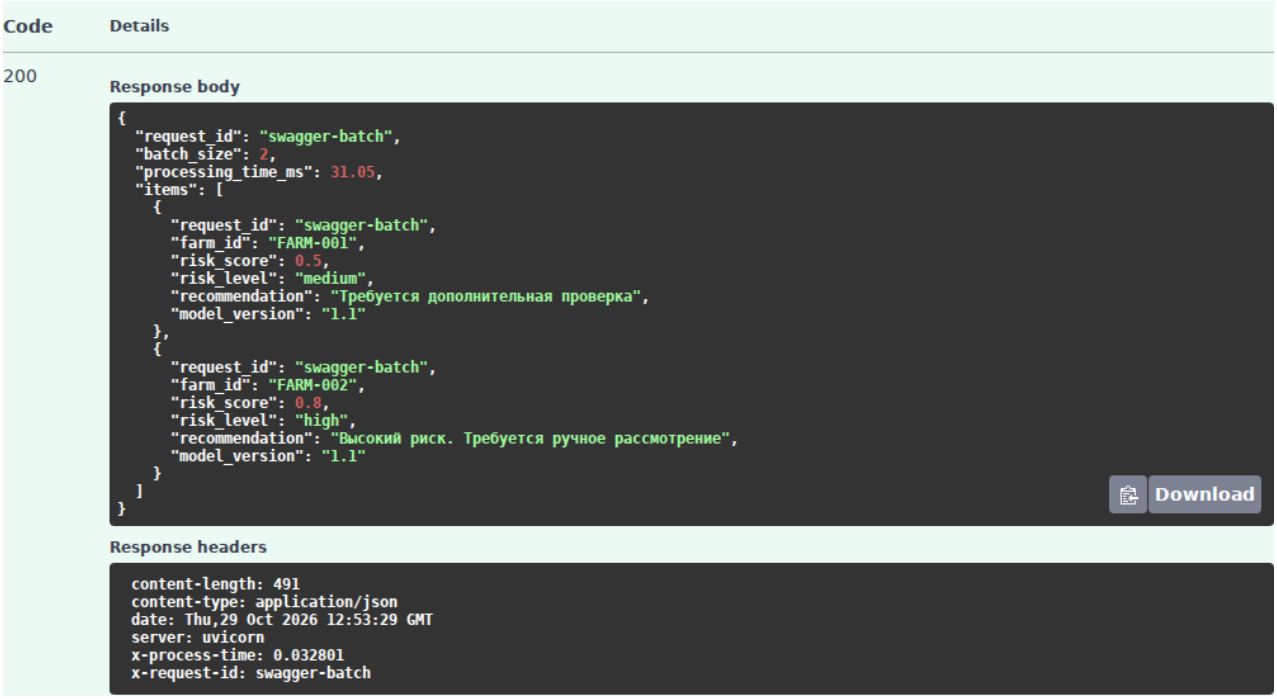


Рис. 5: POST /batch-predict: два хозяйства, один идентификатор запроса

Почему executemany() вместо восьми INSERT + COMMIT? Одна транзакция уменьшает расходы на фиксацию и дисковые синхронизации, а также исключает сохранение только части пакета при SQL-ошибке. При отмене или ошибке до commit соединение закрывается без фиксации.

Очередь ограничена 100 ожидающими элементами. `put_nowait()` позволяет немедленно вернуть 429 при заполнении. Параметры и текущий размер доступны через `GET/queue-status`.

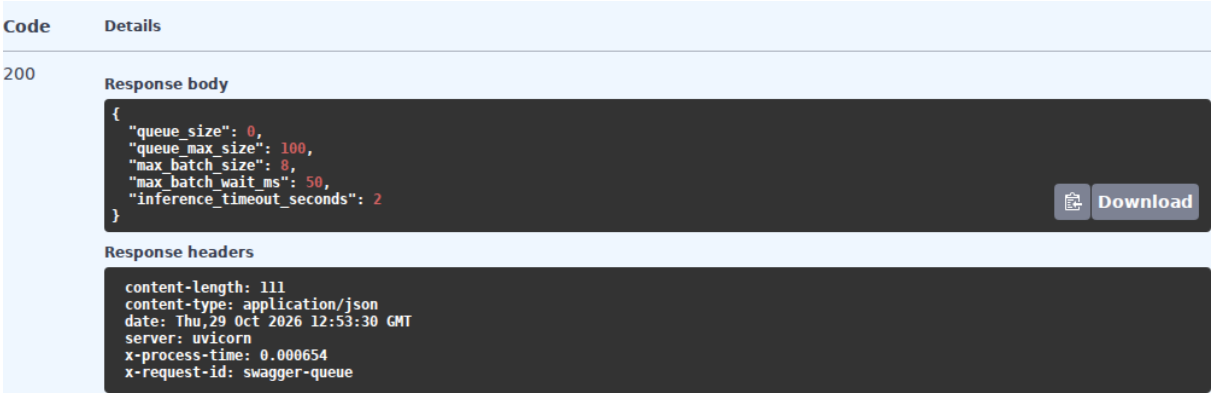


Рис. 6: GET /queue-status: очередь пуста, ограничения соответствуют ТЗ

8. Динамическая обработка и таймаут (задания 9–11)

Клиент `POST/predict-queued` отправляет одно хозяйство. В очередь помещаются данные, request_id и Future. Worker получает первый элемент и собирает пакет до восьми элементов либо до истечения 50 мс с момента первого получения. Затем выполняются общий инференс и общая транзакция. Каждому Future устанавливается результат соответствующего элемента, поэтому параллельные клиенты не получают чужие прогнозы.

В проверке восемь конкурентных запросов объединены в один вызов модели; идентификаторы и farm_id всех восьми ответов совпали с исходными. Проверка переполнения временно приостанавливает worker и заполняет очередь ровно 100 элементами: следующий запрос получает 429. Эти условия задаются только в тесте, служебных HTTP-ручек в проекте нет.

Что будет с неограниченной очередью? При превышении входной интенсивности над скоростью обработки очередь растёт вместе с потреблением памяти. Увеличиваются время ожидания и latency; возможна нехватка памяти. Ограничение очереди даёт backpressure, но само по себе не ускоряет модель.

Инференс обёрнут в `asyncio.wait_for(...)`. Обычно таймаут составляет 2 секунды. При временной установке 0.001 секунды модель с имитацией задержки не успевает завершиться, и API возвращает 504. При этом запись не появляется в БД. После восстановления параметра worker продолжает обслуживать запросы.



Рис. 7: POST /predict при таймауте инференса 0.001 с: 504 и request_id

Таймаут относится к вызову модели. Полная задержка queued-запроса также включает ожидание в очереди, окно накопления пакета и сохранение в БД. Время модели учебное: $20+3n$ мс на пакет из n элементов. Это имитация накладных расходов, а не замер производительности обученной ML-модели.

9. LLM-компонент (задания 12–14)

Используется OpenAI-совместимый `POSThttps://vireonix.ai/v1/chat/completions`, модель auto, без ключа и авторизации. Асинхронный httpx-клиент переиспользуется в lifespan, таймаут — 60 секунд. Запрос содержит messages, model, temperature и max_tokens.

`POST/llm/chat` возвращает request_id, requested_model, returned_model, answer, finish_reason и latency_ms. В `/predict-with-llm` сначала рассчитывается и сохраняется прогноз, затем формируется объяснение. Провайдеру передаются только учебные risk_score и risk_level. LLM не пересчитывает оценку и не меняет категорию; при ошибке объяснения сохранённый прогноз остаётся доступен в истории.

Метод проверки снимков. На двух снимках ниже работающий Swagger вызывает реальное локальное приложение, а внешний ответ подаётся через httpx.MockTransport. Модель `test-provider` явно обозначает тестовый ответ. Снимки проверяют формат HTTP-контракта и разделение скоринга и объяснения; результаты отдельных настоящих вызовов приведены на следующей странице.

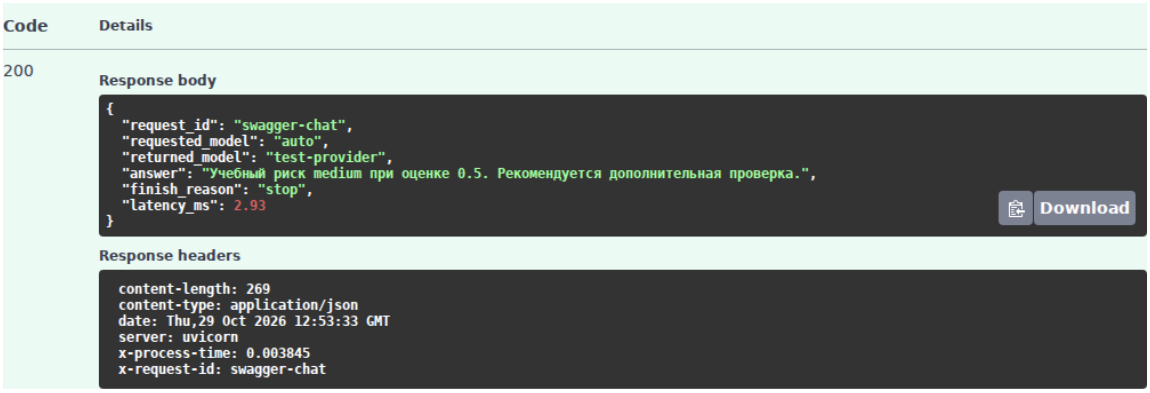


Рис. 8: POST /llm/chat, локальный тестовый ответ провайдера

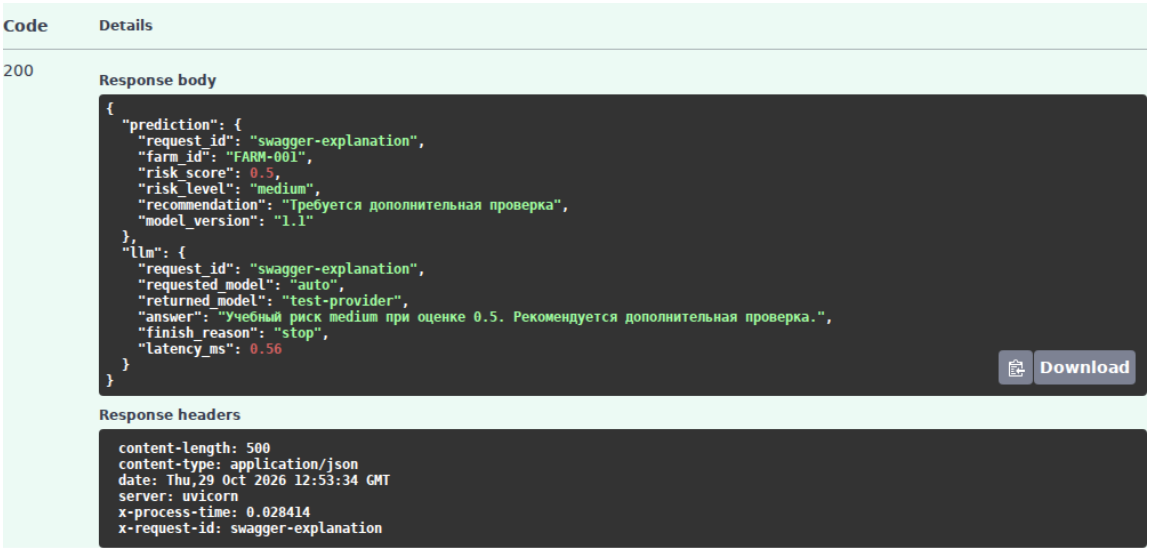


Рис. 9: POST /predict-with-llm, локальный тестовый ответ; оценка 0.5 и medium сохранены

9.1. Настоящие вызовы и сравнение времени

При первичной интеграционной проверке 29 октября 2026 года оба маршрута выполнены с настоящим Vireonix. Данные хозяйства были вымышленными. Ответы и время зафиксированы до повторного снятия иллюстраций с тестовым транспортом.

Маршрут	HTTP	X-Process-Time, с
GET /health	200	0.020612
POST /predict	200	0.027759
POST /llm/chat	200	9.164693
POST /predict-with-llm	200	8.556950

Для `/llm/chat` возвращены модель auto, finish_reason=stop и текст о назначении агроскоринга. В комбинированном ответе сохранены risk_score=0.5 и risk_level=medium. Числа относятся к одному последовательному прогону и не являются нагрузочным benchmark. Внешняя генерация занимает основную часть времени; для /health возможны стартовые накладные расходы.

При недоступном соединении возвращается 503, при таймауте — 504. Ошибочный HTTP-код провайдера преобразуется в 502, кроме HTTP 503. Отсутствие choices, пустой текст или неправильный JSON также дают 502. В автоматических проверках эти ситуации воспроизводятся локально, без зависимости от доступности внешнего сервиса.

10. Тестирование API (задание 15)

Команда `python -m pytest -q`: **22 passed**. Тесты используют временную SQLite и локальный транспорт LLM. Дополнительно выполнены HTTP-вызовы запущенного Uvicorn и работа со Swagger; настоящие вызовы LLM указаны выше. В таблице на следующей странице сохранены все 16 сценариев ТЗ. «Пройден» означает совпадение полученного кода с ожидаемым; проверки аварийных условий выполняются с управляемым отказом зависимости.

Помимо таблицы проверены 409 для пакета, отсутствие частичной записи при невалидном элементе, сохранность после перезапуска, один вызов модели на пакет, сопоставление восьми Future, неизвестный request_id в пакетном чтении, пустая страница, отказ БД и модели, внутренний 500, восстановление worker после 504 и некорректный ответ провайдера.

Таблица 1: Проверка сценариев из ТЗ

№	Запрос / условие	Ожид.	Получ.	Итог
1	GET /health	200	200	Пройден
2	Корректный POST /predict	200	200	Пройден
3	area_ha = -100	422	422	Пройден
4	Неизвестный регион	400	400	Пройден
5	Чтение существующего request_id	200	200	Пройден
6	Чтение неизвестного request_id	404	404	Пройден
7	Корректный POST /batch-predict	200	200	Пройден
8	В batch передано 9 элементов	422	422	Пройден
9	POST /predict-queued	200	200	Пройден
10	Переполненная очередь	429	429	Пройден
11	GET /predictions/by-weekday, page=1, per_page=5	200	200	Пройден
12	page = -1	422	422	Пройден
13	Неизвестный weekday	400	400	Пройден
14	Корректный POST /llm/chat	200	200	Пройден
15	Ошибка HTTP / недоступность / таймаут LLM	502/503/504	502/503/504	Пройден
16	Корректный POST /predict-with-llm	200	200	Пройден

Строки 14 и 16 подтверждены как локальным тестовым транспортом, так и отдельными настоящими вызовами. Для строки 15 отдельно воспроизведены HTTP 500 провайдера (502 клиенту), HTTP 503 и ошибка соединения (503 клиенту), ReadTimeout (504 клиенту). Неправильный формат успешного ответа провайдера также даёт 502.

11. API как контракт (задание 16)

Все успешные коды — 200. Обозначения входа и выхода в таблице: Farm — FarmRequest; Pred — PredictionResponse; Stored — StoredPrediction с id, source и created_at; Batch — BatchRequest/BatchResponse; Page — PaginatedPredictions; Chat — LLMChatRequest/LLMChatResponse; Combined — PredictWithLLMResponse.

Общие для приложения случаи: 500 при непредвиденной ошибке; 422 при неверном теле, заголовке или query-параметре, если такие параметры заданы. Основные предметные ошибки указаны в таблице на следующей странице. Ошибка всегда содержит идентификатор текущего HTTP-запроса. При чтении истории идентификатор в записи относится к исходному запросу, а заголовок ответа — к текущему GET.

Таблица 2: Контракт основных маршрутов

Endpoint	Метод	Input	Output	OK	Ошибки
`/health`	GET	—	status	200	500
`/model-info`	GET	—	Метаданные	200	500
`/predict`	POST	Farm	Pred	200	400, 422, 503, 504
`/batch-predict`	POST	Batch	Batch	200	400, 422, 503, 504
`/predict-queued`	POST	Farm	Pred	200	400, 422, 429, 503, 504
`/queue-status`	GET	—	QueueStatus	200	500
`/predictions`	GET	limit, risk_level	list[Stored]	200	422, 503
`/predictions/by-weekday`	GET	weekday, page, per_page	Page	200	400, 422, 503
`/predictions/{request_id}`	GET	request_id	Stored	200	404, 409, 503
`/requests/{request_id}/predictions`	GET	request_id	list[Stored]	200	404, 503
`/llm/chat`	POST	Chat	Chat	200	422, 502, 503, 504
`/predict-with-llm`	POST	Farm	Combined	200	400, 422, 502, 503, 504

12. Вывод

Сервис расширен без объединения всех компонентов в один файл. request_id связывает HTTP-ответ, журнал и записи БД. SQLite сохраняет историю между запусками. Явный и динамический batching уменьшают число вызовов модели и транзакций. Ограниченная очередь обеспечивает backpressure, таймаут завершает слишком долгий инференс понятным ответом 504.

LLM используется как независимый компонент объяснения и не управляет числовым скорингом. Проверены успешные и ошибочные сценарии, включая отказ внешних зависимостей. Получен работающий учебный API с документированным контрактом и воспроизводимыми автоматическими проверками.